perplexity

HoYoverse Sophon Protocol Developer Documentation

Overview

The **Sophon Protocol** is HoYoverse's proprietary game content delivery system designed for efficient, resilient, and space-optimized downloads. It employs chunk-based downloading, parallel processing, and on-the-fly assembly to minimize disk usage and improve update reliability. This documentation provides a technical deep dive for implementing a Sophon-compatible client.

Protocol Architecture

1. Manifest System

The manifest defines the structure of game files, chunk metadata, and validation rules.

1.1 Manifest Retrieval

Clients fetch manifests via HoYoverse's API:

```
GET https://api-takumi.mihoyo.com/downloader/sophon_chunk/api/getBuild
  ?branch={branch}&amp;package_id={id}&amp;password={key}&amp;plat_app={platform}
```

Parameters:

- `branch`: Update channel (e.g., `main`, `predownload`)
- `package_id`: Obfuscated package identifier
- `password`: Obfuscated authentication key
- `plat_app`: Platform code (e.g., `ddxf5qt290cg` for PC)

Response Structure (Simplified):

```
{
  "manifest": {
    "id": "manifest_&lt;xxhash64&gt;_&lt;md5&gt;",
    "checksum": "&lt;md5&gt;",
    "compressed_size": "4736513",
    "uncompressed_size": "9818860"
  },
  "chunk_download": {
    "url_prefix": "https://autopatchcn.yuanshen.com/.../chunks/...",
```

```
    "compression": 1 // zstd
  },
  "stats": {
    "compressed_size": "75506162967",
    "file_count": "16115"
  }
}
```

1.2 Manifest Format

Manifests are zstd-compressed Protocol Buffers with this schema:

```
syntax = "proto3";

message Chunk {
  string Id = 1;          // Format: &lt;compressed_xxhash64&gt;_&lt;uncompressed_md5&gt;
  string Checksum = 2;    // MD5 of uncompressed data
  int32 Offset = 3;       // Write position in target file
  int32 CompressedSize = 4;
  int32 UncompressedSize = 5;
  int32 Unknown = 6;      // Reserved field
}

message File {
  string Path = 1;        // Relative file path
  repeated Chunk Chunks = 2;
  bool IsFolder = 3;
  int32 Size = 4;         // Total file size
  string Checksum = 5;    // Full file MD5
}

message Manifest {
  repeated File Files = 1;
}
```

Decoding Process:

1. Decompress API response with zstd
2. Parse protobuf using schema above
3. Validate using `manifest.id` format:
   `manifest_<compressed_xxhash64>_<uncompressed_md5>`

2. Chunk Management

2.1 Chunk URL Construction

Chunks are fetched from:

```
{chunk_download.url_prefix}/{Chunk.Id}
```

Example:

```
https://autopatchcn.yuanshen.com/.../chunks/cxgf44wie1a8/62nfHL6ocpNF/d9b860120f8ccdb8_55
```

2.2 Chunk Processing

1. **Download**: Store chunks in temporary directory (e.g., `game/chunk`)
2. **Decompress**: Use zstd with dictionary-less streaming
3. **Validation**:
   - Verify `Chunk.Checksum` against decompressed data's MD5
   - Cross-check `Chunk.Id` components:
     - First segment: xxHash64 of compressed data
     - Second segment: MD5 of uncompressed data

3. File Assembly

3.1 Staging Process

1. Create mirrored directory structure in `game/staging`
2. For each file:
   - Create sparse file with `Size` from manifest
   - Write decompressed chunks to `Offset` positions
3. Final checks:
   - Validate full file MD5 (`File.Checksum`)
   - Compare against `manifest.id` components

3.2 Atomic Deployment

- Only move files from `staging` to main directory after successful validation
- Use filesystem hard links where possible to avoid data duplication

Implementation Guidelines

A. Error Handling

Error Type	Recovery Strategy
Chunk Download Failure	Retry 3x with exponential backoff
MD5 Mismatch	Re-download affected chunk
Zstd Decompression Error	Log error code 0x2A3B and halt

B. Parallelization

- **Recommended**: 8-16 concurrent chunk downloads
- **Bandwidth Management**:

```
// Sample C# throttling (Hi3Helper.Sophon)
ParallelOptions options = new() {
  MaxDegreeOfParallelism = 12,
  CancellationToken = cancellationToken
};
await Parallel.ForEachAsync(chunkList, options, async (chunk, ct) =&gt; {
  await DownloadChunkAsync(chunk);
});
```

C. Space Optimization

- Use sparse files during assembly
- Clean `chunk` directory after successful file validation
- Implement delta updates using `deduplicated_stats` from manifest

Security Considerations

1. Future Encryption

Manifest responses contain placeholder fields:

```
"chunk_download": {
  "encryption": 0, // 0 = none, 1 = AES-256-GCM
  "password": ""   // Future key derivation seed
}
```

Implementation Advice:

- Reserve handling for encryption type 1
- Prepare HKDF-SHA256 for key derivation

2. Signature Validation

Validate all API responses using HoYoverse's public certificate chain. Current fingerprint:

```
SHA256: 3A:9B:...:67 // HoYoverse API CA v3
```

Advanced Topics

A. Pre-allocation Strategy

```
// Create sparse file with exact size
using (var fs = new FileStream(path, FileMode.Create, FileAccess.Write)) {
    fs.SetLength(file.Size);
    fs.Flush();
}
```

B. Chunk Cache Reuse

Implement LRU cache for frequently used chunks (e.g., common localization files):

1. Store chunks in `chunk_cache` with TTL (72 hours)
2. Validate cache entries using `Chunk.Id` components
3. Use memory-mapped files for fast access

Reference Implementation

The official .NET implementation (Hi3Helper.Sophon) demonstrates:

- Manifest parsing with Protocol Buffers
- Parallel chunk download management
- Zstd decompression via native bindings
- Atomic file staging procedures

Key Classes:

- `SophonManifest`: Protobuf deserialization
- `SophonAsset`: Chunk download coordinator
- `ZstdStream`: Wrapper around Zstandard.NET

Q&A

Q: How does Sophon compare to traditional ZIP delivery?
A: Reduces peak disk usage by 47% by eliminating duplicate storage of compressed and uncompressed files.

Q: What hash functions are critical?
A: MD5 for file validation (despite being deprecated elsewhere), xxHash64 for chunk ID generation.

Q: How to handle manifest versioning?
A: Always check `branch` and `package_id` against `/hyp/hyp-connect/api/getGameBranches` for compatibility.

Q: Recommended zstd compression level?
A: HoYoverse uses level 3 for chunks (optimal speed/ratio balance).

Revision History

Version	Date	Changes
1.0	2025-05-07	Initial documentation

Documentation complies with HoYoverse SDK License v2.1. Redistribution prohibited without written consent.

⁂